

Estudio: TRANSLATOR

Índice

1. [¿Qué hace este proyecto?](#)
 2. [Arquitectura general](#)
 3. [Flujo de ejecución](#)
 4. [Desglose de cada archivo](#)
 5. [Dependencias externas](#)
 6. [Conceptos clave](#)
 7. [Historial de mejoras implementadas](#)
 8. [Próximas mejoras propuestas](#)
-

1. ¿Qué hace este proyecto?

Es una **aplicación web de traducción de voz** que permite grabar audio en español y obtener automáticamente audio traducido en 4 idiomas simultáneamente (inglés, francés, portugués, alemán).

Pipeline completo:

1. Graba audio desde el micrófono
 2. Transcribe el audio a texto usando Whisper (OpenAI)
 3. Traduce el texto a 4 idiomas usando Google Translate
 4. Genera audio sintético en cada idioma usando gTTS
 5. Muestra 4 reproductores de audio en la interfaz web
-

2. Arquitectura general

```
main.py ← único archivo (66 líneas)
├── modelo_whisper (módulo) ← carga Whisper "base" una vez al inicio
├── IDIOMAS (dict)           ← mapeo nombre → código de idioma
├── traducir_audio()         ← función principal del pipeline
│   ├── Transcribe: audio → texto (Whisper)
│   ├── Traduce: texto → 4 idiomas (translate)
│   └── TTS: texto → audio (gTTS)
└── gr.Interface             ← interfaz web Gradio
```

```
└─ Input: gr.Audio (micrófono)
└─ Output: 4× gr.Audio (idiomas)
```

Tecnologías involucradas:

| Tecnología | Propósito | Tipo | |---|---|---| | Gradio | Interfaz web interactiva | Librería Python | | Whisper (openai-whisper) | Transcripción de voz a texto | Librería Python | | translate (Google Translate) | Traducción de texto | Librería Python | | gTTS (Google TTS) | Texto a voz sintética | Librería Python | | FFmpeg | Procesamiento de audio (requisito de Whisper) | Binario del sistema |

3. Flujo de ejecución

3.1 Importaciones (líneas 1-5)

```
import gradio as gr          # Framework para interfaces web de ML
import whisper                # Transcripción de voz a texto
from translate import Translator # Traducción (Google Translate)
from gtts import gTTS         # Text-to-Speech (Google)
import os                     # Operaciones del sistema
```

3.2 Carga del modelo Whisper (línea 7)

```
modelo_whisper = whisper.load_model("base")
```

Se ejecuta **una sola vez** al importar el módulo. Whisper "base" pesa ~1.4 GB y tarda varios segundos en cargar.

whisper.load_model(size) : Descarga (si es primera vez) y carga el modelo.
Tamaños disponibles:

Modelo	Parámetros	RAM requerida	--- --- ---
tiny	39M	~1 GB	base
74M	~1 GB	small	244M ~2 GB
medium	769M	~5 GB	large
1550M	~10 GB		

Por qué se carga fuera de la función: Si estuviera dentro de `traducir_audio()`, se cargaría ~1.4 GB en cada llamada, haciendo el proceso extremadamente lento.

3.3 Diccionario de idiomas (líneas 9-14)

```
IDIOMAS = {
    "Inglés": "en",
    "Francés": "fr",
    "Portugués": "pt",
```

```
"Alemán": "de",  
}
```

Clave: Nombre visible en la interfaz (ej: "Inglés"). **Valor:** Código ISO 639-1 para la API de traducción y gTTS.

3.4 La función `traducir_audio()` (líneas 17-45)

3.4.1 Transcripción con Whisper (líneas 18-22)

```
def traducir_audio(audio_file):  
    try:  
        result = modelo_whisper.transcribe(audio_file, fp16=False)  
        transcription = result["text"]  
    except Exception as e:  
        raise gr.Error(f"Error al transcribir el audio: {str(e)}")
```

`modelo_whisper.transcribe(audio_file, fp16=False)` :

- `audio_file` : ruta al archivo de audio (lo proporciona Gradio)
- `fp16=False` : desactiva precisión media (float16). Necesario si no hay GPU compatible.

`result["text"]` : El texto transcrito por Whisper.

`raise gr.Error(...)` : Muestra un error en la interfaz de Gradio (no un crash).

3.4.2 Validación (líneas 24-25)

```
if not transcription.strip():  
    raise gr.Error("No se detectó voz en el audio. Intenta de nuevo.")
```

Si Whisper devuelve texto vacío o solo espacios, se muestra un error en lugar de continuar.

3.4.3 Bucle de traducción y TTS (líneas 27-43)

```
archivos = []  
os.makedirs("audio", exist_ok=True)  
  
for nombre_idioma, lang_code in IDIOMAS.items():  
    try:  
        translator = Translator(from_lang="es", to_lang=lang_code)  
        texto_traducido = translator.translate(transcription)  
    except Exception as e:  
        raise gr.Error(f"Error al traducir a {nombre_idioma}: {str(e)}")
```

```

try:
    save_file_path = f"audio/{lang_code}.mp3"
    tts = gTTS(text=texto_traducido, lang=lang_code)
    tts.save(save_file_path)
    archivos.append(save_file_path)
except Exception as e:
    raise gr.Error(f"Error al generar audio en {nombre_idioma}: {str(e)}")

```

os.makedirs("audio", exist_ok=True) : Crea el directorio si no existe, sin error si ya existe.

Translator(from_lang="es", to_lang=lang_code) : Crea un traductor de español al idioma destino.

translator.translate(transcription) : Traduce el texto vía Google Translate (requiere internet).

gTTS(text=texto_traducido, lang=lang_code) : Genera audio sintético en el idioma destino.

tts.save(save_file_path) : Guarda el audio como archivo MP3.

Cada iteración: Traduce y genera audio para un idioma, secuencialmente (4 veces).

3.5 Interfaz Gradio (líneas 48-63)

```

web = gr.Interface(
    fn=traducir_audio,
    inputs=gr.Audio(
        sources=["microphone"],
        type="filepath",
        label="Graba tu voz"
    ),
    outputs=[
        gr.Audio(label="Inglés"),
        gr.Audio(label="Francés"),
        gr.Audio(label="Portugués"),
        gr.Audio(label="Alemán"),
    ],
    title="Traductor de voz",
    description="Traductor de voz con IA a varios idiomas"
)

```

gr.Interface(fn, inputs, outputs) : Crea una interfaz web completa con:

- **fn** : la función Python que procesa los datos
- **inputs** : componente(s) de entrada
- **outputs** : componente(s) de salida

`gr.Audio(sources=["microphone"], type="filepath") :`

- `sources` : permite solo micrófono (no subida de archivos)
- `type="filepath"` : pasa la ruta del archivo temporal a la función

`gr.Audio(label=...)` : 4 salidas de audio, una por idioma.

3.6 Punto de entrada (líneas 65-66)

```
if __name__ == "__main__":  
    web.launch()
```

`web.launch()` : Inicia el servidor web de Gradio en `http://localhost:7860` .

4. Desglose de cada archivo

4.1 `main.py`

Propósito: Único archivo ejecutable. Contiene toda la aplicación. **Líneas:** 66
Responsabilidades:

- Cargar modelo Whisper (línea 7)
- Pipeline de traducción (líneas 17-45)
- Interfaz web (líneas 48-66)

4.2 `requirements.txt`

Propósito: Dependencias para `pip install` .

- `openai-whisper>=20231117`: Transcripción de voz (Whisper)
- `translate>=3.6.1`: Traducción de texto (Google Translate)
- `gradio>=5.0.0`: Interfaz web
- `gtts>=2.5.0`: Text-to-speech (Google)

4.3 `.gitignore`

Excluye: `.venv/` , `.env` , `__pycache__/` , `*.pyc` , `.gradio/` , `audio/`

4.4 `README.md`

Propósito: Documentación del proyecto.

5. Dependencias externas

5.1 FFmpeg (requisito del sistema)

Whisper requiere FFmpeg para procesar audio. No es una librería Python, es un binario del sistema.

Propósito: Convertir entre formatos de audio, recortar, cambiar velocidad, etc.

5.2 openai-whisper

Modelo de transcripción de OpenAI. Descarga el modelo la primera vez que se usa.

Flujo interno:

1. Recibe archivo de audio (cualquier formato gracias a FFmpeg)
2. Divide en segmentos de 30 segundos
3. Pasa cada segmento por la red neuronal
4. Devuelve texto transcrito con timestamps

5.3 translate

Wrapper de Python para Google Translate (no oficial). Envía peticiones HTTP a translate.googleapis.com.

Límite: Google Translate tiene límites de uso no documentados. Para uso intensivo, considerar Google Cloud Translation API.

5.4 gTTS

Google Text-to-Speech. Envía texto a Google y recibe audio MP3.

Idiomas: Soporta la mayoría de idiomas ISO 639-1.

5.5 Gradio

Framework para crear interfaces web interactivas para modelos de ML.

gr.Interface : Crea automáticamente:

- Página web con inputs/outputs
 - Manejo de archivos temporales
 - Cola de peticiones
 - Modo `share=True` para crear enlace público temporal
-

6. Conceptos clave

6.1 Pipeline de procesamiento

Audio → [Whisper] → Texto español → [Translate] ×4 → Textos traducidos → [gTTS]

Cada etapa es independiente y tiene su propia fuente de error:

1. Whisper puede fallar si el audio es de baja calidad
2. Google Translate puede fallar sin internet
3. gTTS puede fallar si el texto tiene caracteres no soportados

6.2 Errores manejados vs no manejados

| Etapa | Error manejado | Consecuencia | |---|---|---| | Transcripción | try/except genérico | Muestra error en UI | | Traducción | try/except por idioma | Muestra qué idioma falló | | TTS | try/except por idioma | Muestra qué idioma falló |

6.3 Whisper vs APIs cloud

| Característica | Whisper local | API OpenAI Whisper | |---|---|---| | Privacidad | Total (todo local) | Datos enviados a OpenAI | | Costo | Gratis (uso eléctrico) | Por minuto de audio | | Velocidad | Según GPU/CPU | Rápido (servidores dedicados) | | Latencia inicial | Alta (carga modelo 1.4GB) | Baja (API siempre lista) |

7. Historial de mejoras implementadas

✓ Mejora 1 — Ruta absoluta → relativa

Archivo: `main.py` **Qué cambió:** `save_file_path` de `/home/alejandro-fuentes/...` a `audio/{lang_code}.mp3` **Por qué:** La ruta absoluta solo funcionaba en la máquina original. Ahora funciona en cualquier computadora.

✓ Mejora 2 — Whisper cargado una sola vez

Archivo: `main.py` **Qué cambió:** `whisper.load_model("base")` movido de dentro de la función a nivel de módulo **Por qué:** Se cargaba ~1.4 GB en cada llamada. Ahora carga una vez al iniciar.

✓ Mejora 3 — `if __name__ == "__main__":`

Archivo: `main.py` **Qué cambió:** `web.launch()` envuelto en el guard **Por qué:** Sin esto, importar el módulo iniciaba el servidor.

✓ Mejora 4 — Código limpiado

Archivo: `main.py`, `requirements.txt` **Qué cambió:** Eliminados `dotenv`, `typer`, `rich` (no se usaban) **Por qué:** Código muerto que confundía.

✓ Mejora 5 — README profesional

Archivo: `README.md` (nuevo) **Qué cambió:** De no tener README a tener documentación completa **Por qué:** Sin README no se podía saber qué hacía el proyecto.

8. Próximas mejoras propuestas

| Prioridad | Mejora | Esfuerzo | Descripción | |---|---|---|---| | **Alta** | Selector de idioma origen | Bajo | Permitir elegir desde qué idioma se traduce | | **Alta** | Selector de idiomas destino | Bajo | Checkboxes para elegir qué idiomas generar | | **Alta** | Subida de archivos + micrófono | Bajo | Agregar `sources=["microphone", "upload"]` | | **Alta** | Tests con pytest | Medio | Tests para transcripción, traducción, export | | **Media** | Limpieza automática de audios | Bajo | Eliminar archivos temporales después de un tiempo | | **Media** | Historial de traducciones | Medio | Guardar traducciones anteriores en la UI | | **Media** | Dockerfile | Bajo | Crear imagen Docker con todo incluido | | **Baja** | API REST | Alto | Exponer endpoint HTTP para traducción | | **Baja** | Soporte GPU | Bajo | Detectar GPU y usar `fp16=True` si está disponible |

¿Qué opinas? ¿Empezamos con las mejoras o quieres revisar el PDF de estudio primero?